

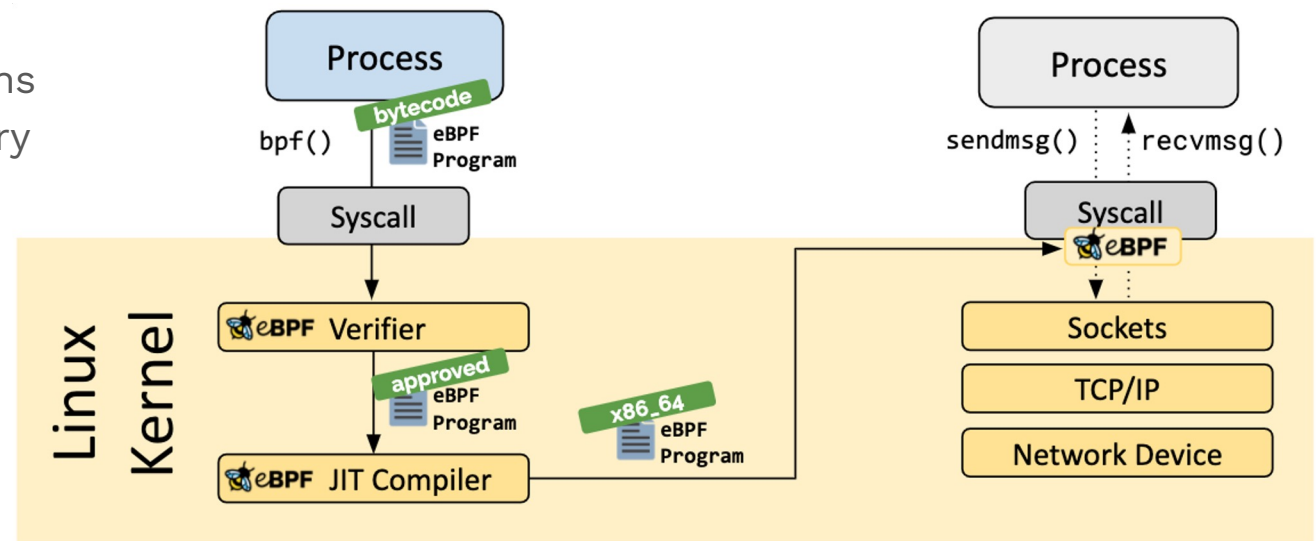
Proof-Carrying-BPF Programs for the eBPF Verifier

Yalu Cai and **Santosh Nagarakatte**
Rutgers University

@eBPF 2026, Prague, Czechia, September 29, 2026

eBPF Verifier Guarantees Safety with Abstract Interpretation

- Running arbitrary user code in the kernel. Security?
- Kernel's solution: statically prove safety of the program
- Some properties to ensure safety
 - Termination
 - No illegal operations
 - Safe kernel memory accesses



Agni: Automated Verification Tools for use as part of Linux Kernel's Continuous Integration Testing Infrastructure

[CGO 2022, CAV 2023, SAS 2024, SAS 2025]

Common Frustration: “Fighting the Verifier”

My eBPF Program Compiled with Clang... but the Kernel Verifier Rejected It



Nishant

Follow

2 min read · Mar 13, 2026

How to Debug and Troubleshoot eBPF Programs

A practical guide to debugging common eBPF program issues and verifier errors.

Debugging eBPF Programs
When the Verifier Says No:
Reading Rejections, State
Explosion, and Helper

Why does the kernel reject safe programs?
Intervals and tristate numbers
overapproximate $\Rightarrow$ safe programs are
rejected!

Concrete Example of a Safe Program Rejection

PROGRAM · BPF ISA

```
P0: R0 = call helper() ; R0 = unknown
P1: R0 &= 15           ; R0 in [0,15]
P2: R1 = R10           ; frame pointer
P3: R1 += -16          ; base of 16B buf
P4: R1 += R0            ; R1 = base + R0
P5: R2 = 15
P6: R2 -= R0            ; R2 = 15 - R0
P7: if R2 < 4 goto P11
P8: R1 += 4             ; base + R0 + 4
P9: R0 = *(u8*)(R1)     ; READ #1
P10: exit
P11: R1 += R2            ; base + 15
P12: R0 = *(u8*)(R1)    ; READ #2
P13: R0 = 0
P14: exit
```

R0 is unknown

R0 = [0, 15]

R1 = base + 0

R1 = [base + 0, base + 15]

R2 = [0, 15]

R1 = [base+4, base + 19]

Out of bounds according to interval bounds!

Concrete Example of a Safe Program Rejection

PROGRAM · BPF ISA

```
P0:  R0 = call helper()    ; R0 = unknown
P1:  R0 &= 15              ; R0 in [0,15]
P2:  R1 = R10              ; frame pointer
P3:  R1 += -16             ; base of 16B buf
P4:  R1 += R0              ; R1 = base + R0
P5:  R2 = 15
P6:  R2 -= R0              ; R2 = 15 - R0
P7:  if R2 < 4 goto P11
P8:  R1 += 4              ; base + R0 + 4
P9:  R0 = *(u8*)(R1)       ; READ #1
P10: exit
P11: R1 += R2              ; base + 15
P12: R0 = *(u8*)(R1)       ; READ #2
P13: R0 = 0
P14: exit
```

A 16-byte stack buffer; valid offsets are 0...15.

The code keeps $R2 = 15 - R0$, so both reads are in bounds.

$R2 \geq 4 \Rightarrow 15 - R0 \geq 4 \Rightarrow R0 \leq 11$

So the READ#1 is safe

$R1 = [\text{base} + 0, \text{base} + 11] + 4 = [\text{base} + 0, \text{base} + 15]$

Symbolic reasoning proves both reads safe, but that reasoning is exactly what the kernel domains can't do.

Can we increase the class of programs
that kernel accepts without increasing
the code it trusts?

This Paper: Proof Carrying BPF Programs

Idea #1:

Design a BPF backend that generates bytecode guaranteed to pass the verifier

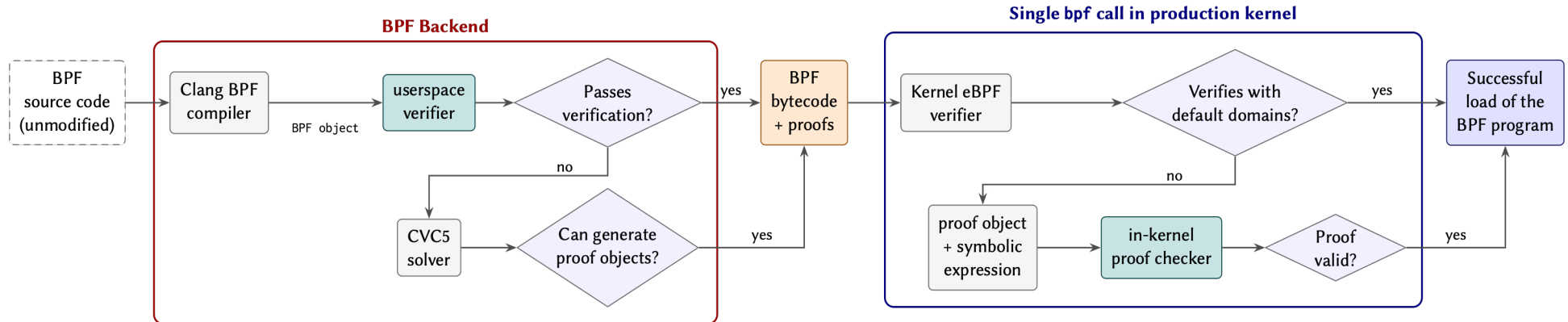
Create a mirror of the kernel verifier in userspace!

Idea #2:

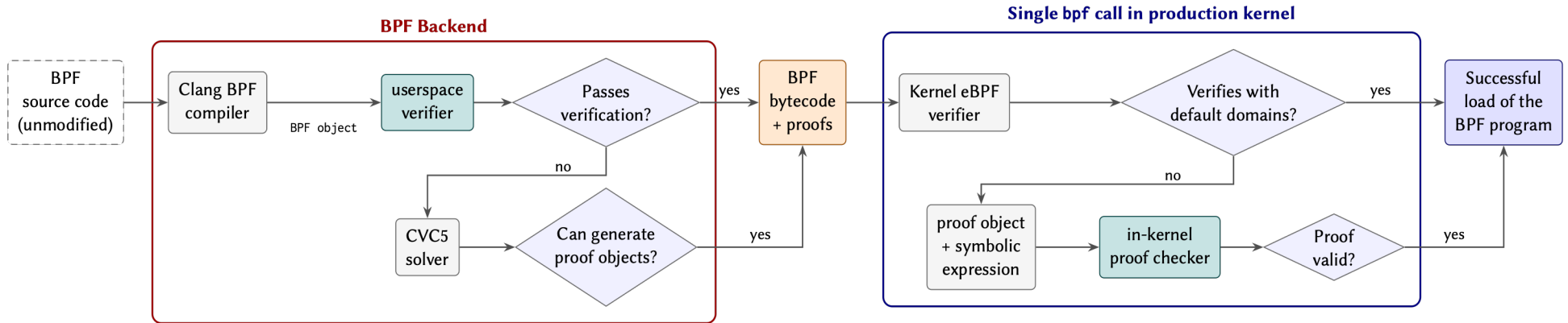
Extend the class of programs accepted by the verifier with richer analysis

The backend generates proofs where the kernel verifier rejects and add in-kernel proof checker

Our PCB Framework



Our PCB Framework



- Mirrors the kernel's default abstract domains.
- At each point the kernel would reject, it forms the refinement condition and solves it with CVC5.
- Stores each (condition, proof) in a hash table, keyed by a canonical hash of the condition.
- Emits all proofs in one pass, bundled with the bytecode.
- Loads bytecode + proof bundle in a single `BPF_PROG_LOAD`.
- At a failing check it reconstructs the condition, hashes it, and finds the matching entry.
- Confirms the condition matches, validates the proof with the small in-kernel checker, continues.

Adding an in-kernel proof checker is much simpler than doing complicated analysis or using non-relational domains in the Kernel!

End-to-End Example: Identify the Slice of Variables

1. Backward pass

2. Forward pass

3. Obligation

4. Proof object

5. Kernel check

PROGRAM · BPF ISA

```
P0:  R0 = call helper() ; R0 = unknown
P1:  R0 &= 15           ; R0 in [0,15]
P2:  R1 = R10           ; frame pointer
P3:  R1 += -16          ; base of 16B buf
P4:  R1 += R0           ; R1 = base + R0
P5:  R2 = 15
P6:  R2 -= R0           ; R2 = 15 - R0
P7:  if R2 < 4 goto P11
P8:  R1 += 4            ; base + R0 + 4
P9:  R0 = *(u8*)(R1)    ; READ #1
P10: exit
P11: R1 += R2           ; base + 15
P12: R0 = *(u8*)(R1)    ; READ #2
P13: R0 = 0
P14: exit
```

Continue until the live set empties

reverse walk	live set after
P4 R1 += R0	{ R1, R0 }
P3 R1 += -16	{ R1, R0 }
P2 R1 = R10 (leaf)	{ R0 }
P1 R0 &= 15	{ R0 }
P0 R0 = call (leaf)	{ } -> Slice ends



Two leaves: the frame pointer R10 (the known buffer base) and the call result. The slice ends at **P0**.

End-to-End Example: Build Symbolic Expressions

✓ Backward pass

2. Forward pass

3. Obligation

4. Proof object

5. Kernel check

PROGRAM · BPF ISA

```
P0:  R0 = call helper() ; R0 = unknown
P1:  R0 &= 15           ; R0 in [0,15]
P2:  R1 = R10           ; frame pointer
P3:  R1 += -16          ; base of 16B buf
P4:  R1 += R0           ; R1 = base + R0
P5:  R2 = 15
P6:  R2 -= R0           ; R2 = 15 - R0
P7:  if R2 < 4 goto P11
P8:  R1 += 4            ; base + R0 + 4
P9:  R0 = *(u8*)(R1)    ; READ #1
P10: exit
P11: R1 += R2           ; base + 15
P12: R0 = *(u8*)(R1)    ; READ #2
P13: R0 = 0
P14: exit
```

Replay P0→P9; the call result becomes sym

insn	R0	R2	R1 offset
P0 call	sym	-	-
P1 R0 &= 15	sym & 15	-	-
P3 R1 += -16			0 (base)
P4 R1 += R0			R0
P5 R2 = 15		15	
P6 R2 -= R0		15 - (sym & 15)	
P8 R1 += 4			(sym & 15) + 4
P9 read			at (sym & 15) + 4



The relation survives between R2 and R0 survives

End-to-End Example: Construct Proof Obligations

✓ Backward pass

✓ Forward pass

3. Obligation

4. Proof object

5. Kernel check

PROGRAM · BPF ISA

```
P0: R0 = call helper() ; R0 = unknown
P1: R0 &= 15           ; R0 in [0,15]
P2: R1 = R10           ; frame pointer
P3: R1 += -16          ; base of 16B buf
P4: R1 += R0           ; R1 = base + R0
P5: R2 = 15
P6: R2 -= R0           ; R2 = 15 - R0
P7: if R2 < 4 goto P11
P8: R1 += 4            ; base + R0 + 4
P9: R0 = *(u8*)(R1)    ; READ #1
P10: exit
P11: R1 += R2          ; base + 15
P12: R0 = *(u8*)(R1)   ; READ #2
P13: R0 = 0
P14: exit
```

PATH (P7 fall-through, “<” reversed)
 $R2 \geq 4$

ACCESS OUT OF BOUNDS ($R0 + 4 > 15$)
 $R0 > 11$

OBLIGATION (refute this $\Rightarrow$ read is safe)
 $(R2 \geq 4) \wedge (R0 > 11)$



UNSATISFIABLE $\rightarrow$ READ #1 is always safe here

Mask by 15 $\Rightarrow R0 \leq 15$. With $R0 > 11$, $R0 \in \{12,13,14,15\}$, so $R2 = 15 - R0 \leq 3$, which contradicts $R2 \geq 4$.

End-to-End Example: The Proof Object

✓ Backward pass

✓ Forward pass

✓ Obligation

4. Proof object

5. Kernel check

ARENA · expressions the proof cites

- 0 `sym`
- 1 `sym & 15` (`R0`)
- 2 `15 - (sym & 15)` (`R2`)
- 3 `R2 ≥ 4` (path)
- 4 `R0 > 11` (out of bounds)
- 5 `id3 ∧ id4` (obligation)

PROOF STEPS (each line derives a fact)

ASSUME assume the obligation: $(R2 \geq 4) \wedge (R0 > 11)$

AND_ELIM $R2 \geq 4$ (the path holds) *from step 0*

AND_ELIM $R0 > 11$ (the read is out of bounds) *from step 0*

BITBLAST $R0 \leq 15$ (masking by 15 clears bits > 3)

POLY_NORM $R2 \leq 3 = \neg(R2 \geq 4)$ ($R2 = 15 - R0$, $R0 \geq 12$) *from 2, 3*

CONTRA $R2 \geq 4$ and $\neg(R2 \geq 4) \Rightarrow \text{false}$ *from 1, 4*

End-to-End Example: Deriving the Contradiction

✓ Backward pass

✓ Forward pass

✓ Obligation

4. Proof object

5. Kernel check

THE THREE JOINING STEPS

BITBLAST — **bits**. Masking by 15 (1111) forces $R0 \leq 15$.

POLY_NORM — **arithmetic**. The key join: $R0 \geq 12$, $R0 \leq 15$, $R2 = 15 - R0 \Rightarrow R2 \leq 3$ — the negation of the path.

CONTRA — **finish**. Holding $R2 \geq 4$ and its negation yields false.

PROOF STEPS (all facts established)

ASSUME

assume the obligation: $(R2 \geq 4) \wedge (R0 > 11)$

AND_ELIM

$R2 \geq 4$ (the path holds)

from step 0

AND_ELIM

$R0 > 11$ (the read is out of bounds)

from step 0

BITBLAST

$R0 \leq 15$ (masking by 15 clears bits > 3)

POLY_NORM

$R2 \leq 3 = \neg(R2 \geq 4)$ ($R2 = 15 - R0$, $R0 \geq 12$)

from 2, 3

CONTRA

false refutation complete

End-to-End Example: Kernel Just Checks the Proof

✓ Backward pass

✓ Forward pass

✓ Obligation

✓ Proof object

5. Kernel check



THE KERNEL SAVED

$(R2 \geq 4) \wedge (R0 > 11)$

The only thing the kernel trusts is the small in-kernel proof checker, never the userspace solver.

REPLAY · re-derive & verify each step

kernel re-derives & verifies	accepts fact
assumed obligation matches the saved one	$(R2 \geq 4) \wedge (R0 > 11)$
$R2 \geq 4$ really is one conjunct	$R2 \geq 4$
$R0 > 11$ really is the other conjunct	$R0 > 11$
masking by 15 really forces $R0 \leq 15$	$R0 \leq 15$
with $R0 \geq 12$, $R0 \leq 15$, $R2 = 15 - R0 \Rightarrow R2 < 4$	$\neg(R2 \geq 4)$
step 4 negates step 1 $\Rightarrow$ false	false



Final fact = false $\Rightarrow$ proof accepted. The verifier tightens $R1$'s offset ($R0 \leq 11$) and continues verification with the next instruction.

Experimental Evaluation

Is the backend verifier faithful to the Kernel verifier?

FAITHFULNESS

Kernel selftests

3,267

0 false accepts

0 false rejects

More Programs Accepted and Loaded Faster

360 production programs
now accepted —
previously rejected

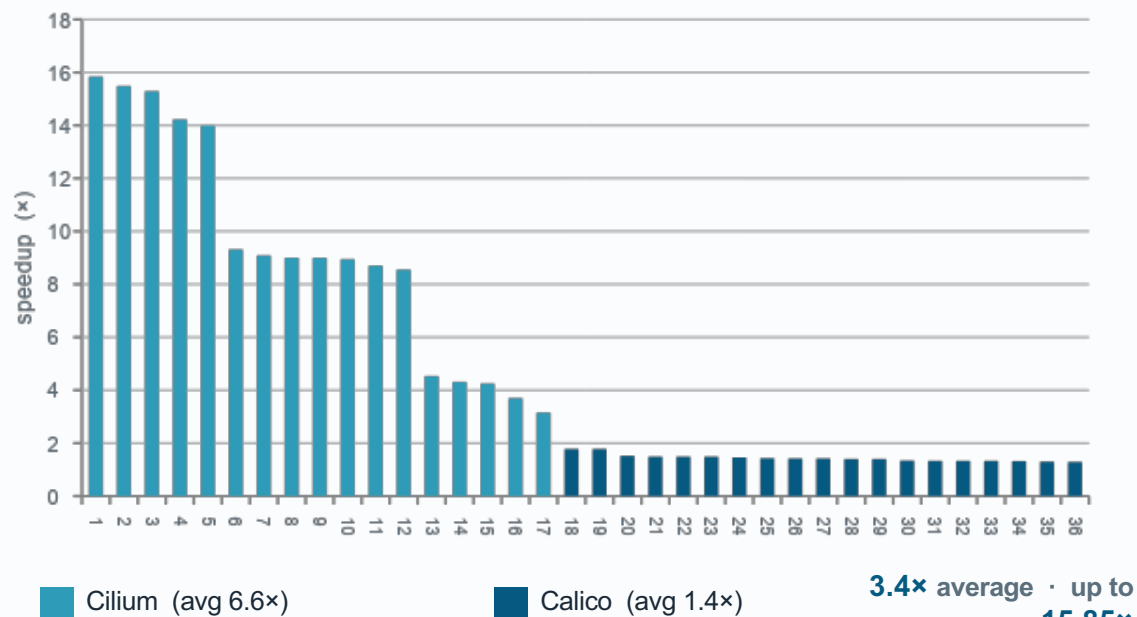
337 Calico

17 Cilium TC

3 IG seccomp

3 BCC

Load-time speedup over BCF (36 sample programs)



Related Work

Proof-carrying code

Necula & Lee, OSDI 96

Whole-program proofs against a kernel policy. PCB applies the principle to eBPF but carries proofs only where the domains fall short.

ExoBPF

Nelson et al., LPC'21

Replaces the verifier with a Lean proof checker. PCB keeps the verifier in place and needs far fewer kernel changes.

BCF

SOSP'25

Same trust model, but proves on demand with kernel↔userspace round-trips. PCB proves everything ahead of time which yields 3.4× faster loads.

PREVAIL

eBPF-on-Windows

A relational (zone) userspace verifier. Adopting it in Linux means a bigger analyzer or trusting an external tool.

REX

safe-Rust extensions

Avoids the verifier via the Rust toolchain + runtime. PCB keeps the verifier as final judge and eBPF as the unit of loading.

Conclusion

- A case for a BPF backend that not only generates code but also proofs.
- Extend the class of programs by generating proofs along with code and couple with an in-kernel checker
- Accepts a large class of programs that were previously rejected by the Kernel verifier with a small TCB

Open Source

Visit the PCB and Alivio GitHub page for details: <https://github.com/rutgers-apl/Alivio>

